System Design Master Roadmap: Topics and Resources

Shashank ke liye: LLD + HLD + core CS ke saare topics ek jagah, saath me resources. Ye ek checklist ki tarah use karo: jo topic khatam ho, uske aage apna tick lagate jao.

Suggested Order

1. Core CS fundamentals (OOP, DBMS, OS, CN), DSA ke saath parallel
2. LLD (OOP, SOLID, design patterns, machine-coding problems)
3. HLD fundamentals
4. Classic HLD problems
5. Apne project me practical implementation (Redis, queue, rate limiting)

Rule: ek time pe ek main resource + ek backup. 10 resources shuru karke koi complete na karna sabse badi galti hai.

Part 1: Core CS Prerequisites

- OOP: abstraction, encapsulation, inheritance, polymorphism, interfaces vs abstract classes, composition vs inheritance
- DBMS: normalization, indexing, transactions, ACID, isolation levels, joins, query optimization basics
- Operating Systems: processes vs threads, context switching, scheduling, memory management, deadlock, synchronization (mutex, semaphore)
- Computer Networks: OSI/TCP-IP model, TCP vs UDP, HTTP/HTTPS, DNS, TLS handshake, sockets

Part 2: Low Level Design (LLD)

Foundations

- UML: class diagrams, sequence diagrams, relationships (association, aggregation, composition, inheritance)
- SOLID principles: SRP, OCP, LSP, ISP, DIP

- Other principles: DRY, KISS, YAGNI, Law of Demeter, composition over inheritance

Design Patterns

- Creational: Singleton, Factory Method, Abstract Factory, Builder, Prototype
- Structural: Adapter, Decorator, Facade, Proxy, Composite, Bridge, Flyweight
- Behavioral: Strategy, Observer, Command, State, Chain of Responsibility, Template Method, Iterator, Mediator, Memento, Visitor
- Priority patterns (interviews me sabse zyada): Strategy, Factory, Singleton, Observer, Decorator, State, Builder, Command, Adapter, Chain of Responsibility

Machine-Coding / LLD Problems

- Must do: Parking Lot, Elevator, Vending Machine, ATM, Tic Tac Toe, Snake and Ladder, Chess
- Must do: LRU Cache, Rate Limiter, Library Management, BookMyShow, Splitwise
- Good to do: Logger framework, Pub-Sub system, Task Scheduler, Notification system, Food delivery app, Ride booking, Music player, Hotel management, Digital wallet

Supporting Topics

- Concurrency: threads, thread pools, locks, deadlock prevention, producer-consumer, thread-safe singleton
- Dependency injection, exception handling, clean code, testing basics
- Database schema design for an LLD problem, basic API design

How to answer an LLD interview

1. Clarify requirements
2. Identify entities and operations
3. Define classes, responsibilities and interfaces
4. Pick patterns only where they solve a real problem
5. Write core code
6. Discuss trade-offs, extensibility and concurrency

Part 3: High Level Design (HLD) Fundamentals

Basics

- Scalability: vertical vs horizontal scaling
- Latency vs throughput, availability, reliability, fault tolerance
- Back-of-envelope estimation: QPS, storage, bandwidth, powers of 2, latency numbers

Networking and APIs

- DNS, CDN, HTTP/HTTPS, TCP vs UDP
- WebSockets, long polling, Server-Sent Events
- REST, GraphQL, gRPC
- API design: idempotency, pagination, versioning, rate limiting, authentication

Traffic Layer

- Load balancing: L4 vs L7, algorithms (round robin, least connections, consistent hashing)
- Reverse proxy, API gateway, forward proxy

Caching

- Strategies: cache-aside, read-through, write-through, write-back, write-around
- Eviction: LRU, LFU, FIFO, TTL
- Redis and Memcached, cache stampede, cache invalidation
- CDN caching

Databases and Storage

- SQL vs NoSQL: key-value, document, column-family, graph
- Indexing: B-tree, LSM tree, secondary indexes
- Replication: leader-follower, multi-leader, leaderless
- Sharding and partitioning, consistent hashing, hot keys
- Transactions, isolation levels, optimistic vs pessimistic locking
- Object and blob storage, file systems
- Search: inverted index, Elasticsearch basics

Distributed Systems Theory

- CAP theorem, PACELC
- Consistency models: strong, eventual, causal
- Consensus: Raft and Paxos basics, leader election
- Distributed locks, gossip protocol, vector clocks, clock synchronization
- Two-phase commit, saga pattern

Asynchronous Systems

- Message queues and streaming: Kafka, RabbitMQ, SQS
- Pub-Sub, event-driven architecture, delivery guarantees (at-most-once, at-least-once, exactly-once)
- Dead letter queues, backpressure, retries with exponential backoff

Architecture Patterns

- Monolith vs microservices, service discovery, service mesh basics
- CQRS, event sourcing
- Circuit breaker, bulkhead, rate limiter, graceful degradation
- Idempotent consumers, outbox pattern

Security

- Authentication vs authorization, sessions vs JWT, OAuth 2.0, SSO
- HTTPS and TLS, encryption at rest and in transit, common attacks (XSS, CSRF, SQL injection, DDoS)

Operations

- Monitoring, logging, tracing, alerting, SLI/SLO/SLA
- CI/CD, blue-green and canary deployments
- Containers and orchestration basics: Docker, Kubernetes

Part 4: Classic HLD Problems

Basic:

- URL shortener, Pastebin, Rate limiter, Unique ID generator, Key-value store

Intermediate:

- Twitter/Instagram news feed, WhatsApp/chat system, Notification system, Typeahead/autocomplete, Web crawler, Ticket booking, Distributed cache

Advanced:

- YouTube/Netflix, Google Drive/Dropbox, Uber/ride-hailing, Google Maps, Payment system, Search engine, Metrics and monitoring system, Distributed message queue, Distributed file system

For each problem, practice this flow:

1. Requirements (functional and non-functional)
2. Estimation
3. API design
4. Data model
5. High-level diagram
6. Deep dive (bottlenecks, scaling, failure handling)
7. Trade-offs

Part 5: Resources

Note: ye sab resources mujhe jaante hain, par kuch ki current availability, pricing aur latest content verify nahi kiya. Link kholke khud ek baar check kar lena.

LLD

- Coder Army LLD Masterclass playlist (Aditya Tandon, YouTube, Hindi): main LLD resource
- Refactoring Guru website: design patterns ke clear explanations aur diagrams
- Head First Design Patterns (book): patterns ke liye beginner-friendly
- Clean Code by Robert C. Martin (book): principles ke liye
- Java Concurrency in Practice (book): concurrency ke liye, agar Java use karte ho

HLD Concepts

- Piyush Garg system design playlist: pehle dekh chuke ho, revision ke liye
- Gaurav Sen (YouTube): concepts clear
- ByteByteGo (YouTube and newsletter): diagrams ke saath

- The System Design Primer (GitHub, donnemartin): free, ek jagah sab kuch
- Striver (takeUforward) system design course: structured format chahiye toh

HLD Problems and Interview Prep

- System Design Interview Vol 1 and Vol 2 by Alex Xu (books)
- Hello Interview (website): system design breakdowns
- Tech Dummies (YouTube): Hindi/English, Indian interview context

Deep Dive (baad me, final year ya jab basics solid hon)

- Designing Data-Intensive Applications (DDIA) by Martin Kleppmann
- Distributed systems lecture series (MIT 6.824)
- Engineering blogs of Netflix, Uber, Meta, Discord, Cloudflare: real-world architecture ke liye

Practice

- Apne projects me Redis caching, rate limiting, ya ek queue daalo
- LLD problems GitHub pe push karo (task manager, pub-sub, parking lot)
- Mock interviews friends ke saath: ek bolta hai design, doosra sawaal poochta hai

Part 6: Rough Timeline (5th sem se)

- Month 1-2: LLD (Coder Army playlist) + OOP revision, DSA parallel
- Month 3: Concurrency + remaining LLD problems (parking lot, elevator, LRU, rate limiter)
- Month 4-5: HLD fundamentals (caching, DB, queues, distributed concepts)
- Month 6-7: Classic HLD problems, har hafte 2 problems
- Ongoing: DSA 60% time, project me practical implementation
- Final year: DDIA, mock interviews, advanced problems